UNIVERSITY OF MUMBAI

MINI PROJECT 1 REPORT

ON

SMART STREET LIGHT SYSTEM

Automatic Light Control with Adaptive Brightness

Submitted in partial fulfilment of the requirements of the Computer Science Project

SUBMITTED BY

Shrikant Ganesh Gadhave

Roll No. 04
Enrollment No. 510512637
Bachelor of Science in Computer Science
Semester V

UNDER THE GUIDANCE OF
Asst.Prof. Nirali Mehta

DEPARTMENT OF COMPUTER SCIENCE
ST. WILFRED'S COLLEGE OF ARTS, COMMERCE AND SCIENCE
(Affiliated to Mumbai University)NAAC 'B++'
Shedung, Panvel, Navi Mumbai, Maharashtra

ACADEMIC YEAR 2026–2027
MONTH OF SUBMISSION: [September, 2026]

ST. WILFRED'S COLLEGE OF ARTS, COMMERCE AND SCIENCE

CERTIFICATE BY THE INSTITUTE

This is to certify that **Mr. Shrikant Ganesh Gadhave** (Roll No. 04), a student of Bachelor of Science in Computer Science, has undertaken the Mini Project titled "Smart Street Light System" during the academic year 2026–2027.

The project report has been prepared as part of the Mini Project 1 component for academic evaluation. The project details and supporting records are subject to verification by the concerned faculty guide.

Internal Guide

Signature: _____

External Examiner

Signature: _____

Head of Department

Signature: _____

Principal

Signature: _____

College Seal: _____

DECLARATION

I declare that this project report titled “Smart Street Light System” has been prepared for academic purposes. The report describes the project concept, hardware arrangement, program behaviour, operating steps and test procedures of an automatic street light controller that uses a light dependent resistor (LDR), a PIR motion sensor, a current sensor and a NodeMCU ESP8266 board.

I understand that all statements about successful operation, sensor accuracy, energy saving, fault-alert response time and cloud notifications must be supported by tests performed on the actual prototype. Any test or measurement table in this report that is left blank must be completed after testing. Photographs and screenshots included in the final submitted copy must show the real project and the actual code used.

I also acknowledge that this classroom prototype is a low-voltage demonstration model. It is not certified lighting equipment and must not be connected to mains-voltage street lighting without proper electrical design, protection and approval by a qualified electrician or engineer.

Student signature:

_____

Shrikant Ganesh Gadhave (Roll No. 04)

Date: _____ Place: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, Asst.Prof. Nirali Mehta, and the Department of Computer Science at St. Wilfred’s College of Arts, Commerce and Science for the opportunity to develop this project. The project gave me practical experience in microcontrollers, sensors, embedded programming, PWM-based brightness control and the principles of Internet of Things (IoT) monitoring.

I also thank my teachers, classmates and family members for their encouragement and support during the planning and preparation of the prototype. Their feedback helped me organise the work into manageable stages, from understanding the street lighting problem to documenting the circuit, software and test process.

Finally, I acknowledge the authors and maintainers of the Arduino and ESP8266 documentation, the sensor-library documentation and the cloud platform guides that help students learn embedded-system development. All references used for the final submission should be checked and formatted according to the college’s preferred citation style.

Shrikant Ganesh Gadhave

ABSTRACT

Conventional street lighting stays switched on at full brightness for a fixed period every night, whether or not anyone is using the road. This wastes electrical energy, raises maintenance costs, and gives no early warning when a lamp fails. This project describes a low-cost academic prototype of a Smart Street Light System that controls the ON/OFF state and the brightness of an LED street light automatically, using the surrounding light level and the presence of vehicles or pedestrians.

The prototype uses a Light Dependent Resistor (LDR) to sense darkness and a Passive Infrared (PIR) sensor to detect movement. A NodeMCU ESP8266 board processes the readings, switches the LED on at a low, energy-saving brightness at dusk, raises it to full brightness through PWM and a MOSFET driver when motion is detected, and dims it again when the road is clear. A current sensor (INA219) measures the load current so that a lamp that should be lit but draws no current is flagged as faulty. The built-in Wi-Fi of the ESP8266 sends the readings to a cloud dashboard for real-time monitoring, fault alerts and energy-usage review.

The report covers the problem identification, feasibility study, requirement engineering, SDLC and SRS, UML modelling (use case, class, flowchart, data flow, sequence, activity, ER and deployment diagrams), system architecture, circuit connections, program logic, user manual, test plan, troubleshooting and source code. Measured results, photographs and screenshots must be added from the real prototype. The system is meant for educational demonstration; any real roadside installation requires certified components, weatherproofing and electrical safety design.

Keywords: Smart Street Light, IoT, LDR, PIR Sensor, NodeMCU ESP8266, PWM Dimming, Energy Efficiency, Automation.

TABLE OF CONTENTS

1. TECHNICAL REPORT AND PROJECT INTRODUCTION 7
2. PROBLEM STATEMENT, MOTIVATION AND OBJECTIVES 7
3. FEASIBILITY STUDY AND PROJECT PROPOSAL 8
4. REQUIREMENT ENGINEERING 9
5. SCOPE, APPLICATIONS AND LIMITATIONS 9
6. HARDWARE COMPONENTS AND SPECIFICATIONS 10
7. SOFTWARE REQUIREMENTS AND DEVELOPMENT TOOLS 11
8. SDLC AND SRS DOCUMENT 11
9. SYSTEM BLOCK DIAGRAM AND ARCHITECTURE 12
10. CIRCUIT CONNECTIONS AND POWER CONSIDERATIONS 14
11. WORKING PRINCIPLE AND PROGRAM LOGIC 14
12. PROGRAM EXPLANATION (ARDUINO / NODEMCU) 15
13. USER MANUAL – INSTALLATION, SETUP AND OPERATION 16
14. TESTING PLAN AND TEST CASES 16
15. RESULTS AND EVALUATION RECORD 17
16. TROUBLESHOOTING AND MAINTENANCE 18
17. SECURITY, SAFETY AND RELIABILITY 18
18. CONCLUSION AND FUTURE IMPROVEMENTS 19
19. PROJECT PHOTOGRAPHIC EVIDENCE 20
19. PROJECT PHOTOGRAPHIC EVIDENCE (CONTINUED) 21
20. SOURCE CODE 22
21. SYSTEM MODELING USING UML 24

1. TECHNICAL REPORT AND PROJECT INTRODUCTION

1.1 Introduction

Street lighting is one of the largest contributors to a municipality's electricity bill. Traditional systems rely on timers, manual switching or simple photocells. These do not take real traffic, weather or moonlight into account, so many lamps burn at full power on empty roads. With the growth of embedded systems and the Internet of Things (IoT), it is now practical to build lighting that senses its environment, adapts to it and reports its own health.

1.2 Project Overview

The proposed prototype uses a NodeMCU ESP8266 development board as the controller. An LDR tells the controller when it is dark. A PIR sensor tells it when a vehicle or pedestrian is present. The controller drives an LED street light through a logic-level MOSFET using PWM, so the light can be off, dim or fully bright. An INA219 current sensor measures the load current to detect a failed lamp. The ESP8266's Wi-Fi uploads status data to a cloud platform such as ThingSpeak, where an administrator can view charts and receive fault alerts.

1.3 Project Outcomes

- A physical sensor-to-controller circuit that switches the street light ON at dusk and OFF in daylight.
- Adaptive brightness: dim when the road is empty and full when motion is detected.
- A current-based fault check that flags a lamp that is commanded ON but draws no current.
- Periodic upload of light level, brightness, motion and current to a cloud dashboard.
- A documented build, upload and test procedure, with space for genuine photographs and measurements.

2. PROBLEM STATEMENT, MOTIVATION AND OBJECTIVES

2.1 Problem Statement

Existing street lighting in most towns operates on fixed timers or simple photocell switches. This causes four practical problems:

- Lights stay at full brightness all night even on roads with almost no traffic, wasting energy.
- Faulty or damaged lights are usually found only when a citizen reports them, which delays repairs.
- There is no central system to monitor the real-time status or energy use of street lights.
- Manual operation and inspection increase labour cost and response time.

The problem addressed by this project is how to build a low-cost controller that switches and dims a street light according to darkness and road usage, detects lamp failure automatically and reports to a remote dashboard.

2.2 Existing System and Its Limitations

The existing manual or timer-based system depends on a person or a fixed schedule to switch lights ON at dusk and OFF at dawn. Photocell-based systems improve on this by sensing darkness, but they still keep the light at full brightness regardless of road usage, and they give no feedback about faults or energy use. There is no remote monitoring, no adaptive brightness and no historical data for planning maintenance.

2.3 Motivation

The motivation is to design a cost-effective, energy-efficient and low-maintenance street lighting solution that adapts to the environment and to road usage. By combining sensors, a microcontroller and IoT connectivity, the system reduces wasted energy, improves road safety through adaptive brightness, and allows remote monitoring and fault detection – key needs of modern smart-city infrastructure.

2.4 Objectives

- To switch street lights ON/OFF automatically based on ambient light intensity.
- To adjust brightness dynamically when vehicles or pedestrians are present.
- To detect faulty lights and send real-time alerts to maintenance staff.
- To monitor and log energy-related data through a web/mobile dashboard.
- To reduce overall power consumption and manual intervention in street light operation.

3. FEASIBILITY STUDY AND PROJECT PROPOSAL

3.1 Proposed System

The proposed Smart Street Light System overcomes the limitations above by combining an LDR (darkness), a PIR sensor (vehicles or pedestrians), a current sensor (faults) and a Wi-Fi-enabled microcontroller. The light is dimmed when no movement is detected and brightened when needed, while status is reported continuously to a cloud dashboard.

Aspect	Existing system	Proposed system
Switching	Fixed timer or simple photocell	LDR threshold with hysteresis, controlled by NodeMCU
Brightness	Always full when ON	Off / dim / full, based on PIR motion
Fault detection	Reported manually by citizens	Automatic, from load current
Monitoring	None	Wi-Fi upload to a cloud dashboard
Energy data	Not available	Logged and charted over time

3.2 Feasibility Study

Technical feasibility. The project uses easily available, low-cost parts (NodeMCU, LDR, PIR sensor, INA219, logic-level MOSFET, LED) and free tools (Arduino IDE, ThingSpeak). The skills needed – embedded C/C++, basic electronics and IoT connectivity – are within the scope of an undergraduate mini project.

Economic feasibility. The hardware cost per unit is low compared with the long-term saving from adaptive dimming and with the reduced inspection effort from early fault detection, so the idea is economically viable for larger deployment.

Operational feasibility. Most functions are automatic, so little training is needed. The dashboard gives administrators and maintenance staff a simple view of status and alerts.

Schedule feasibility. With a well-defined scope, requirement analysis, design, assembly, coding, testing and documentation fit within one academic semester.

3.3 Project Proposal Summary

Attribute	Description
Project title	Smart Street Light System
Domain	Internet of Things (IoT) / Embedded Systems
Objective	Automated, energy-efficient street light control with remote monitoring
Target users	Municipal bodies, smart-city administrators, maintenance staff, students
Expected outcome	Lower energy waste, faster fault detection, central monitoring
Duration	One academic semester

4. REQUIREMENT ENGINEERING

4.1 Requirement Gathering

Requirements were gathered by studying existing street lighting, reviewing published IoT-based lighting solutions, and discussing the expected functions and achievable scope with the project guide. They were then classified into functional and non-functional requirements, together with the hardware and software needed.

4.2 Functional Requirements

- FR1: Turn the street light ON when ambient light falls below a set threshold.
- FR2: Turn the street light OFF when ambient light rises above the threshold (daytime).
- FR3: Detect motion of vehicles or pedestrians with a PIR sensor and raise the brightness.
- FR4: Dim the light to a low, energy-saving level when no motion is detected for a set period.
- FR5: Detect a fault (no current drawn when the light should be ON) and generate an alert.
- FR6: Send status and sensor data to a cloud server at regular intervals.
- FR7: Let an administrator view real-time status and history on a dashboard.

4.3 Non-Functional Requirements

Attribute	Requirement
Reliability	Operate continuously with minimal downtime and no false triggering (hysteresis on the LDR).
Performance	Sensor reading and light control in near real time.
Scalability	More light units can be added without redesign (one unit per pole, one channel or device each).
Energy efficiency	Lower energy use than a light that is always fully ON.
Usability	Simple, graphical dashboard for non-technical staff.
Maintainability	Hardware and software modules are easy to replace, update or debug.

5. SCOPE, APPLICATIONS AND LIMITATIONS

5.1 Scope

The initial scope is a single-pole educational prototype on a breadboard or small board. It covers the NodeMCU ESP8266, an LDR, a PIR sensor, an INA219 current sensor, a MOSFET-driven LED and Wi-Fi upload to a cloud dashboard. The core control logic (ON/OFF, dim, full) must keep working locally even if Wi-Fi is unavailable.

5.2 Possible Application Areas

Area	Potential role	Important qualification
Residential colony roads	Dim at night, full on movement	Prototype only; needs certified mains design
College / campus paths	Adaptive lighting with fault alerts	Needs weatherproof enclosure
Parking lots	Light when vehicles or people arrive	PIR range must be tested for the area
Smart-city pilots	Central monitoring of many poles	Needs secure network and scale testing
Classroom / laboratory	Teach sensors, PWM and IoT	Use low voltage, with supervision

5.3 Limitations

- The LDR and PIR are low-cost sensors with limited range and accuracy; they need calibration for each site.
- A single PIR may miss slow or distant objects and can be triggered by animals.
- The prototype is low voltage; switching mains-powered lamps needs proper isolation and approval.

- Cloud features depend on Wi-Fi/GSM coverage and a configured service (free plans have update-rate limits).
- A power failure stops the controller unless a battery or solar backup is provided.
- Threshold values chosen for the demonstration are not universal lighting standards.

6. HARDWARE COMPONENTS AND SPECIFICATIONS

Component	Purpose	Key notes
NodeMCU ESP8266 (ESP-12E)	Central controller with built-in Wi-Fi	3.3 V logic, one 10-bit ADC (A0), PWM on GPIO pins, USB power
LDR with 10 k ohm resistor	Detects day/night	Voltage divider read on A0 (0–1023)
PIR sensor (HC-SR501 type)	Detects motion	Needs 30–60 s warm-up; output HIGH on motion; adjustable delay/sensitivity
INA219 current sensor	Measures load current for fault detection	I2C interface (SDA/SCL), 3.3 V compatible, reports current in mA
Logic-level N-MOSFET (e.g. IRLZ44N)	Switches and dims the LED using PWM	Gate through 220 ohm; 10 k ohm gate pull-down
LED street light module	Output lamp	Low-voltage LED (for example 5–12 V) with suitable series resistor or driver
Power supply / solar panel with battery	Powers the unit	USB 5 V for NodeMCU; separate LED supply with common ground
Breadboard, jumper wires	Assembly without soldering	Check supply and ground rails before power-up

6.1 NodeMCU ESP8266 Development Board

The NodeMCU is the controller. It reads the LDR on its analogue input, reads the PIR on a digital input, produces a PWM signal for the MOSFET, talks to the INA219 over I2C and connects to Wi-Fi to upload data. Its logic level is 3.3 V, so 5 V signals must not be applied directly to its GPIO pins.

6.2 LDR (Light Dependent Resistor)

The resistance of an LDR falls as light increases. Connected as a voltage divider with a fixed resistor, it gives a voltage that rises with brightness. The ADC converts this to a value between 0 and 1023. Two thresholds (one to switch ON, a higher one to switch OFF) create hysteresis so the light does not flicker at dusk.

6.3 PIR Motion Sensor

The PIR sensor detects changes in infrared radiation caused by a moving warm body such as a person or a vehicle engine. Its output goes HIGH while motion is detected. It needs a warm-up period after power-up, and its range and hold-time are adjustable with the two small potentiometers on the module.

6.4 INA219 Current Sensor

The INA219 measures the voltage drop across a small shunt resistor placed in series with the LED supply. The NodeMCU reads the current over I2C. If the light is commanded ON but the current is almost zero, the lamp or wiring is faulty.

6.5 MOSFET Driver and LED

A logic-level MOSFET acts as an electronic switch. PWM on its gate changes the average power to the LED, which sets the brightness. The LED supply is separate from the NodeMCU’s 3.3 V, but the grounds must be joined.

6.6 Power Supply and Solar Option

The prototype can run from a USB adapter. In a deployed unit, a solar panel with a charge controller and battery can provide off-grid power, which supports the energy-saving aim of the project.

7. SOFTWARE REQUIREMENTS AND DEVELOPMENT TOOLS

Software	Purpose
Arduino IDE (with ESP8266 board package)	Write, compile and upload the C/C++ firmware to the NodeMCU
Embedded C / C++	Programming language for the control logic
Adafruit INA219 library	Read current from the INA219 over I2C
ESP8266WiFi and ESP8266HTTPClient	Connect to Wi-Fi and send HTTP requests
ThingSpeak / Firebase / Blynk	Cloud storage, charts and alerts (ThingSpeak is used in the sample code)
HTML / CSS / JavaScript	Optional custom web dashboard
Proteus / Fritzing (optional)	Circuit design and simulation

7.1 Sketch Structure

The firmware follows the standard Arduino structure: `setup()` configures pins, PWM, I2C, the current sensor and Wi-Fi once; `loop()` repeatedly samples the sensors, updates the light level, checks for faults and uploads data on a timer. Timing uses `millis()` rather than `delay()` so the controller never blocks.

7.2 Configuration Values

Constant	Meaning	Sample value
DARK_ON_LEVEL / DARK_OFF_LEVEL	LDR readings that mean “dark” and “bright” (hysteresis)	300 / 400
PWM_DIM / PWM_FULL	Brightness levels (0–1023)	256 / 1023
IDLE_TIMEOUT_MS	Time with no motion before dimming	10000 ms
FAULT_DELAY_MS	Settling time before a fault is declared	3000 ms
MIN_CURRENT_MA	Current below which a lit lamp is faulty	5.0 mA (tune to the LED)
UPLOAD_EVERY_MS	Cloud update interval	20000 ms

8. SDLC AND SRS DOCUMENT

8.1 SDLC Model Used

The Incremental / Iterative Waterfall Model was adopted. The requirements (sensing, control, monitoring) were well understood at the start, so Waterfall gives a clear, structured path. An incremental approach was used across hardware integration, control logic and dashboard development so that each module could be tested on its own before final integration.

8.2 Phases of SDLC Followed

Phase	Activity carried out
Requirement analysis	Identified functional and non-functional needs; prepared the SRS
System design	Designed the architecture, UML diagrams and circuit
Implementation	Assembled hardware, wrote embedded code, integrated modules
Testing	Tested sensors, control logic and dashboard connectivity
Deployment	Installed and demonstrated the working prototype
Maintenance	Fault-alert mechanism planned for ongoing upkeep

8.3 Software Requirement Specification (SRS)

Purpose. This SRS describes the functional and non-functional requirements of the Smart Street Light System and serves as a reference for design, development and testing.

Scope. The system controls street light operation automatically from ambient light and motion, detects faults and reports status to a remote dashboard, aiming to cut energy use and manual maintenance effort.

Intended audience. Project guide, evaluators and developers.

Product perspective. A self-contained embedded IoT unit attached to each street light pole, communicating with a cloud server that gathers data from many units.

Product functions. Automatic ON/OFF control, adaptive brightness, fault detection, data logging and remote monitoring.

User characteristics. Administrators and maintenance staff who may not be technical; the dashboard is simple and graphical.

Assumptions and dependencies. A stable power supply and network (Wi-Fi/GSM); sensor accuracy within manufacturer tolerances.

External interfaces. *Hardware:* sensors, MOSFET and current sensor connected to NodeMCU GPIO, ADC and I2C pins. *Software:* firmware talks to the cloud over HTTP (or MQTT). *User:* web/mobile dashboard showing light status, usage charts and fault alerts.

9. SYSTEM BLOCK DIAGRAM AND ARCHITECTURE

The architecture has four layers: the Sensor Layer (data acquisition), the Control Layer (microcontroller processing), the Communication Layer (Wi-Fi/GSM) and the Application Layer (cloud server and dashboard), supported by an Actuator Layer and a Power Layer. Figure 9.1 shows how they interact.

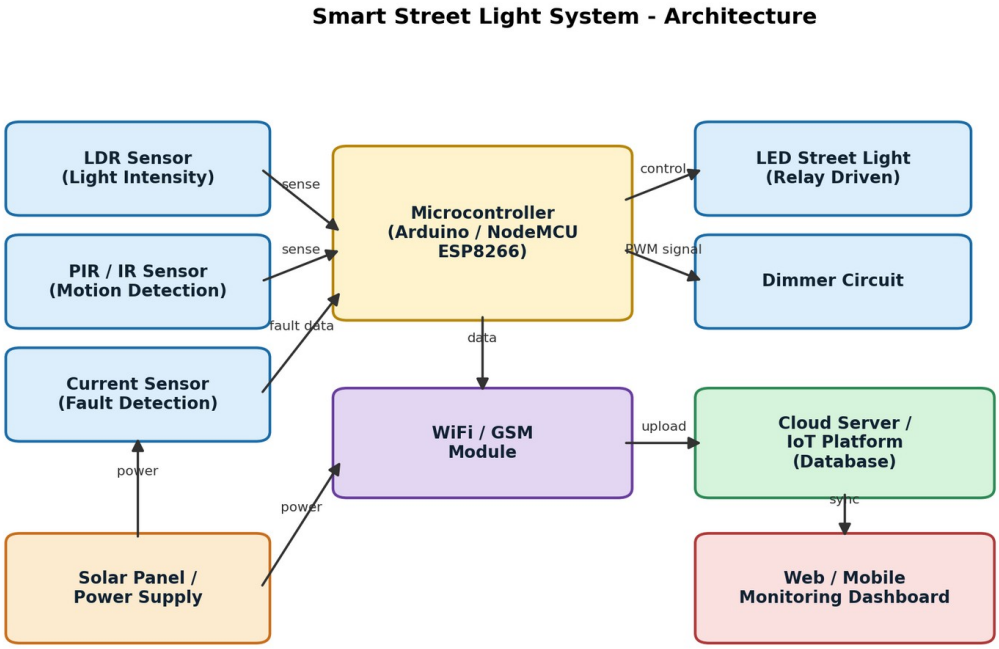


Figure 9.1: System Architecture Diagram

9.1 Description of Layers

- **Sensor layer:** LDR, PIR and current sensor capture ambient light, motion and load data.
- **Control layer:** the microcontroller processes inputs and applies the control logic.
- **Actuator layer:** the MOSFET/dimmer drives the LED according to the controller's PWM output.
- **Communication layer:** the ESP8266 Wi-Fi (or a GSM module) sends status to the cloud.
- **Application layer:** the cloud stores data and powers the dashboard, reports and alerts.
- **Power layer:** solar panel with battery backup, or mains-derived supply, powers the unit.

9.2 Data Flow and Local Independence

Sensor data flows from the sensors to the NodeMCU, which decides the light level and sends status to the cloud. The control decision is taken locally, so the street light still switches and dims correctly if the network is down; only monitoring and alerts are affected.

10. CIRCUIT CONNECTIONS AND POWER CONSIDERATIONS

Module pin	NodeMCU pin	Remarks
LDR divider midpoint	A0	LDR to 3V3, 10 k ohm to GND; midpoint to A0
PIR OUT	D5 (GPIO14)	PIR VCC to VIN (5 V), GND to GND; output is 3.3 V logic
MOSFET gate	D6 (GPIO12)	Through 220 ohm; 10 k ohm from gate to GND
MOSFET source / drain	GND / LED cathode	LED anode to LED supply positive via INA219
INA219 SDA	D2 (GPIO4)	I2C data
INA219 SCL	D1 (GPIO5)	I2C clock
INA219 VCC / GND	3V3 / GND	VIN+ to LED supply +, VIN- to LED anode
Common ground	GND	Join NodeMCU GND, LED supply GND, sensors GND

The ESP8266 has only one analogue input, so the LDR uses it and the current is read digitally over I2C with the INA219. This avoids sharing A0 between two analogue sensors.

10.1 Power-Up Checklist

- Disconnect power while wiring and check each connection against the table above.
- Check that no 5 V signal reaches a NodeMCU GPIO pin.
- Confirm the MOSFET orientation (gate, drain, source) from its datasheet.
- Confirm the LED supply voltage and the LED’s current rating before connecting.
- Apply power and watch the Serial Monitor for start-up messages.

10.2 Common Ground

The NodeMCU and the LED supply must share a ground. Without it, the MOSFET gate signal has no reference and the light may not switch or may flicker.

11. WORKING PRINCIPLE AND PROGRAM LOGIC

11.1 Initialisation

At start-up the NodeMCU sets the pin modes, sets the PWM range, starts I2C and the INA219, opens the Serial port and attempts to join Wi-Fi. The PIR needs about a minute to settle, so early motion readings may be ignored.

11.2 Reading and Darkness Decision

Every 200 ms the controller reads the LDR. If the reading falls below the lower threshold, it decides it is dark; it returns to daylight only when the reading rises above the higher threshold.

11.3 Adaptive Brightness

When dark, the light turns ON at the dim level. If the PIR output is HIGH, the brightness goes to full and the idle timer restarts. When no motion has been seen for the idle timeout, the light returns to the dim level. In daylight the light is OFF.

11.4 Fault Detection

If the light has been ON for longer than the settling delay and the measured current is below the minimum current, a fault is flagged and sent to the dashboard. It clears when current returns to normal or the light is switched OFF.

11.5 Data Upload

Every 20 seconds the controller sends the LDR value, brightness percentage, motion state, current and fault flag to the cloud. If Wi-Fi is down, it tries to reconnect and the lighting logic continues unchanged.

11.6 Pseudocode

```
INITIALISE pins, PWM, I2C, INA219, Serial, Wi-Fi
REPEAT every 200 ms:
    ldr    = read LDR (0-1023)
    motion = read PIR
    current = read INA219 (mA)
    IF ldr < DARK_ON_LEVEL THEN dark = TRUE
    IF ldr > DARK_OFF_LEVEL THEN dark = FALSE
    IF dark = FALSE:
        light OFF
    ELSE:
        IF motion: light FULL, restart idle timer
        ELSE IF light is OFF: light DIM
        ELSE IF light is FULL and idle timer expired: light DIM
    IF light ON for > FAULT_DELAY and current < MIN_CURRENT:
        fault = TRUE
    ELSE: fault = FALSE
EVERY 20 s: upload ldr, brightness, motion, current, fault
```

12. PROGRAM EXPLANATION (ARDUINO / NODEMCU)

12.1 Important Code Sections

- **Configuration block:** Wi-Fi name and password, ThingSpeak write key, pin map and tunable constants.
- **setBrightness():** writes the PWM value to the MOSFET and notes when the light was first switched ON.
- **readSensors():** reads the LDR, PIR and INA219.
- **updateLight():** darkness hysteresis and the OFF / DIM / FULL decision.
- **checkFault():** compares current with the minimum after the settling delay.
- **uploadData():** builds the ThingSpeak HTTP request and sends it.
- **loop():** runs the sampling and upload timers using millis().

12.2 Changing the Thresholds

Open the Serial Monitor to see the LDR value in a dark room and in daylight, then set DARK_ON_LEVEL a little above the dark value and DARK_OFF_LEVEL a little above that. Change IDLE_TIMEOUT_MS to make the light stay bright for longer or shorter after motion.

12.3 Tuning the Fault Threshold

Measure the current of your LED at the dim level and at full brightness, then set MIN_CURRENT_MA below the smaller of the two. If the dim current is too small for the INA219 to read reliably, raise the dim PWM level or use a shunt that suits your LED current.

12.4 Cloud Setup Note

Create a ThingSpeak channel with five fields (LDR, brightness %, motion, current mA, fault) and paste its Write API key into the sketch. The free ThingSpeak plan accepts updates only at limited intervals, which is why the sample uploads every 20 seconds. Fault alerts can be configured in ThingSpeak using its React or TalkBack-style features, or in Blynk or Firebase if preferred.

13. USER MANUAL – INSTALLATION, SETUP AND OPERATION

13.1 Install the Arduino IDE and Board Package

- Install the Arduino IDE on your computer.
- In File > Preferences, add the ESP8266 board manager URL, then open Tools > Board > Boards Manager and install “esp8266”.
- Select Tools > Board > NodeMCU 1.0 (ESP-12E Module) and the correct COM port.

13.2 Install Libraries

In Sketch > Include Library > Manage Libraries, install “Adafruit INA219” (and its dependency Adafruit BusIO when asked). The Wi-Fi and HTTP client libraries come with the ESP8266 package.

13.3 Assemble the Circuit

- With power disconnected, wire the circuit using the table in Chapter 10.
- Check the MOSFET pins, LED polarity and the common ground.
- Place the LDR where it receives ambient light, but not light from the street lamp itself.
- Aim the PIR at the road area to be monitored.

13.4 Configure, Compile and Upload

- Open the sketch from Chapter 20 in the Arduino IDE.
- Enter your Wi-Fi name, password and ThingSpeak Write API key.
- Click Verify, then Upload. Open the Serial Monitor at 115200 baud.

13.5 Normal Operation

- **Daylight:** light is OFF.
- **Dark, no motion:** light glows at the dim level.
- **Dark, motion:** light goes to full brightness and returns to dim after the idle timeout.
- **Fault:** if the lit lamp draws no current, the Serial Monitor shows FAULT and the dashboard flags it.

13.6 Safe Test Method

Test the dark condition by covering the LDR with a hand or a small opaque cap. Test motion by waving a hand in front of the PIR. Test the fault alert by briefly disconnecting the LED while the light is ON. Use only low voltage during testing.

14. TESTING PLAN AND TEST CASES

The tests below check each module separately and then the whole system. The last column must be completed with the real observed result and a Pass/Fail mark after testing on the actual prototype.

ID	Test case	Condition	Expected result	Actual / Status
T1	Power-up	Connect USB power	Serial Monitor shows start-up and Wi-Fi status	
T2	LDR reading	Room lit, then covered	Value drops when covered	
T3	Darkness ON	LDR covered	Light turns ON at dim level	
T4	Daylight OFF	LDR uncovered	Light turns OFF	
T5	Hysteresis	Light level near the threshold	No flicker	
T6	PIR response	Wave hand in dark	Light goes to full brightness	
T7	Idle dimming	No motion for idle timeout	Light returns to dim	

ID	Test case	Condition	Expected result	Actual / Status
T8	Current reading	Light ON	INA219 current is non-zero	
T9	Fault detection	Disconnect LED while ON	FAULT shown and flag sent	
T10	Fault clear	Reconnect LED	Fault flag clears	
T11	Cloud upload	Wi-Fi connected	Data appears in the dashboard	
T12	Network loss	Switch off the router	Lighting logic continues; upload resumes later	

14.1 Unit and Integration Testing

Unit tests (T2, T6, T8) check each sensor alone, using Serial Monitor prints. Integration tests (T3 to T10) check that the sensors and the MOSFET driver work together under the control logic. System tests (T11, T12) check cloud upload and behaviour when the network fails.

15. RESULTS AND EVALUATION RECORD

This chapter is for recording what was actually observed on the prototype. Do not copy values from other projects.

15.1 Observed Light Levels and Behaviour

Condition	LDR reading	Light state	Brightness (%)	Current (mA)
Daylight				
Dark, no motion				
Dark, motion				
Fault (LED disconnected)				

15.2 Energy Comparison (to be measured)

Mode	Duration (min)	Load energy (mWh)	Remarks
Always full brightness			
Smart mode (dim + motion)			

15.3 Evaluation Questions

- Did the light switch ON and OFF at the expected light levels?
- Did the brightness rise promptly on motion and return to dim after the timeout?
- Did a disconnected LED produce a fault flag within a few seconds?
- Did the dashboard receive and chart the data?
- How much energy was saved compared with always-full operation in the same test period?

16. TROUBLESHOOTING AND MAINTENANCE

Problem	Possible cause	What to check
Light never turns ON	Threshold too low; LDR wiring	Print LDR value; adjust DARK_ON_LEVEL; check divider
Light flickers at dusk	Thresholds too close	Increase gap between DARK_ON_LEVEL and DARK_OFF_LEVEL
Light does not brighten on motion	PIR warm-up; wiring; sensitivity	Wait 60 s; check D5 and 5 V supply; adjust PIR pots
Always full brightness	PIR hold time too long or false triggers	Reduce PIR delay; shield from heat sources
False fault alerts	MIN_CURRENT_MA too high; INA219 wiring	Measure real current; check shunt connection
No data in cloud	Wi-Fi, key or rate limit	Check SSID/password, API key and 15 s minimum interval
NodeMCU keeps resetting	Weak USB supply or short circuit	Use a better cable/adapter; check wiring
LED flickers or stays off	Missing common ground; wrong MOSFET pins	Join all grounds; check gate/drain/source

16.1 Maintenance

- Clean the LDR window and PIR lens periodically.
- Check connections and the enclosure for moisture and corrosion.
- Recheck LDR thresholds when the site lighting changes.
- Update the firmware and cloud key when credentials change.

17. SECURITY, SAFETY AND RELIABILITY

17.1 Electrical Safety

Keep the prototype low voltage. Use correct polarity, a current-limited LED supply and insulated wiring. Do not connect mains voltage to the breadboard. A deployed system needs isolation, fuses/MCBs, surge protection and weatherproof enclosures designed by a qualified person.

17.2 Software Reliability

The control logic uses non-blocking timing, hysteresis and local decision-making. A watchdog and automatic Wi-Fi reconnect help recover from faults. A safe default (for example, dim ON in the dark) is better than leaving the road unlit if a sensor fails.

17.3 Security

- Do not publish Wi-Fi passwords or API keys in the report or public repositories.
- Use HTTPS or a secure MQTT broker for real deployments and protect the dashboard with a login.
- Keep the firmware updated and restrict who can change thresholds.

17.4 Operational Limitation

This is an educational prototype. Road safety lighting is regulated; any real deployment must follow local lighting standards and electrical codes.

18. CONCLUSION AND FUTURE IMPROVEMENTS

18.1 Conclusion

The Smart Street Light System shows how simple sensors, a low-cost microcontroller and IoT connectivity can improve conventional street lighting. By adjusting brightness from ambient light and motion, the design aims to reduce unnecessary energy use, while fault detection and remote monitoring can shorten maintenance response time. A structured approach – feasibility study, requirement engineering, SRS, UML modelling and architecture design – led to a well-planned, functionally clear prototype. Actual energy savings must be confirmed with measurements from the built prototype.

18.2 Future Improvements

- Solar power with battery storage for fully off-grid operation.
- Machine learning to predict traffic patterns and optimise brightness schedules.
- Weather and traffic sensors for adaptive lighting in fog or rain.
- A mesh network among street lights to reduce dependence on individual Wi-Fi/GSM links.
- A mobile app with push notifications for instant fault alerts.
- Several PIR sensors or radar sensors for better coverage along the road.

18.3 References

[1] Arduino documentation – https://www.arduino.cc/

[2] ESP8266 Arduino Core documentation – https://arduino-esp8266.readthedocs.io/

[3] Adafruit INA219 library – https://github.com/adafruit/Adafruit_INA219

[4] ThingSpeak documentation – https://www.mathworks.com/help/thingspeak/

[5] I. Sommerville, “Software Engineering”, 10th Edition, Pearson Education.

[6] R. S. Pressman, “Software Engineering: A Practitioner’s Approach”, McGraw-Hill Education.

[7] Datasheets of the LDR, PIR sensor, INA219 and MOSFET used in the project.

19. PROJECT PHOTOGRAPHIC EVIDENCE

Insert genuine photographs and screenshots of the final prototype in the boxes below before submission.

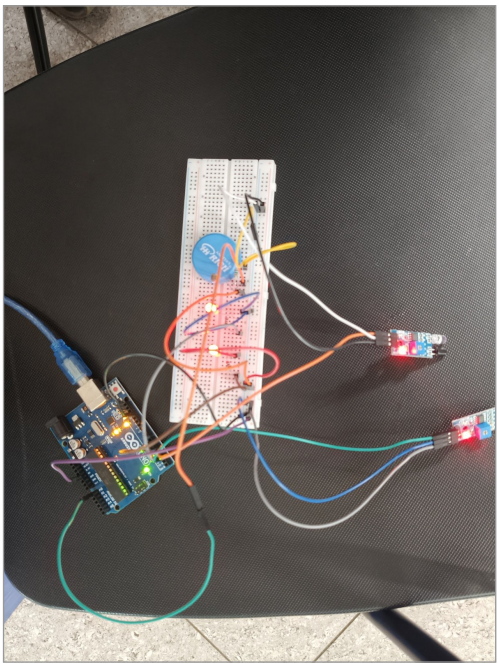


Figure 19.1: Assembled Smart Street Light prototype (Arduino Uno, sensor modules and breadboard)

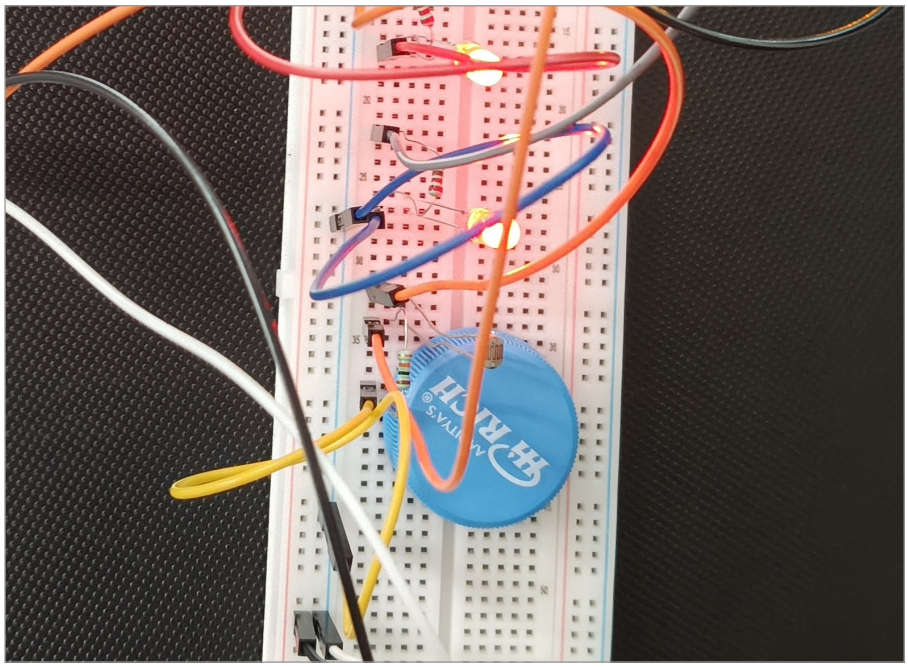


Figure 19.2: Close-up of breadboard wiring with LDR, resistors and LEDs

19. PROJECT PHOTOGRAPHIC EVIDENCE (CONTINUED)

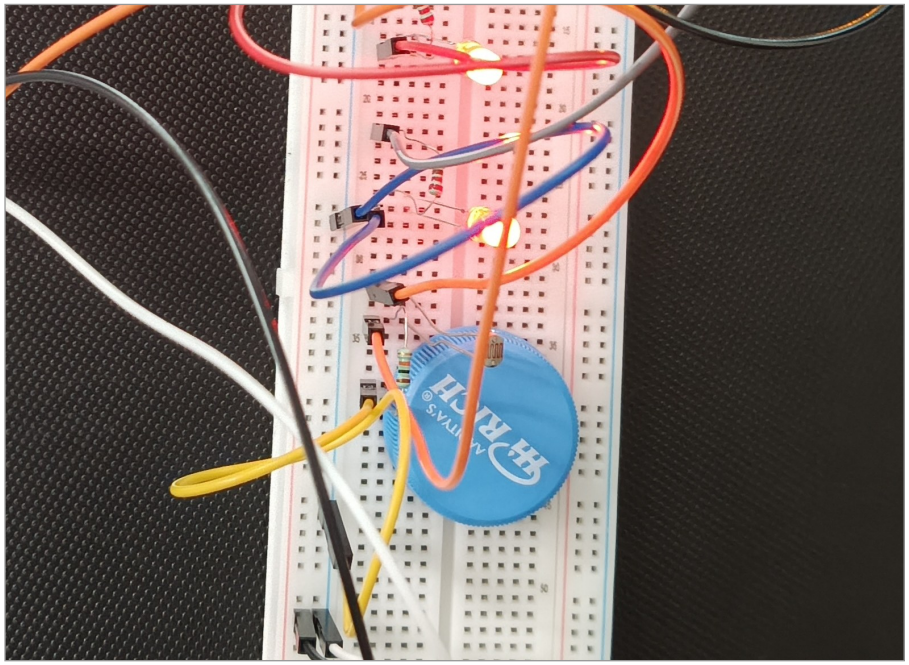


Figure 19.3: Light in dim state (dark, no motion) and full state (motion detected)

20. SOURCE CODE

The complete Arduino sketch for the NodeMCU ESP8266 is given below. Replace the three configuration strings before uploading, and replace this listing with a screenshot of your final tested code if your college requires it.

```
/*
  Smart Street Light System - NodeMCU ESP8266 (Arduino IDE)
  Student : Shrikant Ganesh Gadhave (Roll No. 04)
  Guide   : Asst.Prof. Nirali Mehta
  Function: LDR darkness -> dim light; PIR motion -> full light;
            INA219 current -> fault flag; Wi-Fi upload to ThingSpeak.
*/
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>
#include <Wire.h>
#include <Adafruit_INA219.h>

// ---------- User configuration ----------
const char* WIFI_SSID  = "YOUR_WIFI_NAME";
const char* WIFI_PASS  = "YOUR_WIFI_PASSWORD";
const char* TS_API_KEY = "YOUR_THINGSPEAK_WRITE_API_KEY";

// ---------- Pin map (NodeMCU labels) ----------
const int LDR_PIN = A0;   // LDR + 10k divider, reads 0-1023 (higher = brighter)
const int PIR_PIN = D5;   // PIR output, HIGH when motion
const int LED_PIN = D6;   // PWM to MOSFET gate
const int SDA_PIN = D2;   // INA219 SDA
const int SCL_PIN = D1;   // INA219 SCL

// ---------- Tunable values ----------
const int DARK_ON_LEVEL  = 300;     // below this -> it is dark
const int DARK_OFF_LEVEL = 400;     // above this -> it is bright
const int PWM_OFF  = 0;
const int PWM_DIM  = 256;           // about 25 percent
const int PWM_FULL = 1023;          // 100 percent
const unsigned long IDLE_TIMEOUT_MS = 10000UL;  // dim after 10 s without motion
const unsigned long FAULT_DELAY_MS  = 3000UL;   // settle time before fault check
const float MIN_CURRENT_MA = 5.0;               // tune to your LED
const unsigned long SAMPLE_EVERY_MS = 200UL;
const unsigned long UPLOAD_EVERY_MS = 20000UL;  // ThingSpeak free plan limit

// ---------- State ----------
Adafruit_INA219 ina219;
bool inaOk = false;
bool isDark = false;
bool faultActive = false;
bool motion = false;
int  ldrValue = 0;
int  pwmNow = PWM_OFF;
float currentMa = 0.0;
unsigned long lastMotionMs = 0;
unsigned long lightOnSinceMs = 0;
unsigned long lastSampleMs = 0;
unsigned long lastUploadMs = 0;

void setBrightness(int pwm) {
  if (pwmNow == PWM_OFF && pwm > PWM_OFF) {
    lightOnSinceMs = millis();           // light has just been switched ON
  }
  pwmNow = pwm;
  analogWrite(LED_PIN, pwmNow);
}

void connectWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("Connecting to Wi-Fi");
  unsigned long start = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - start < 15000UL) {
    delay(500);
    Serial.print(".");
  }
  Serial.println(WiFi.status() == WL_CONNECTED ? " connected" : " failed (will retry)");
}

void readSensors() {
```

```
  ldrValue = analogRead(LDR_PIN);
  motion = (digitalRead(PIR_PIN) == HIGH);
  if (inaOk) {
    currentMa = ina219.getCurrent_mA();
  }
}

void updateLight(unsigned long now) {
  // darkness with hysteresis
  if (ldrValue < DARK_ON_LEVEL)  isDark = true;
  if (ldrValue > DARK_OFF_LEVEL) isDark = false;

  if (!isDark) {
    setBrightness(PWM_OFF);
    return;
  }
  if (motion) {
    lastMotionMs = now;
    setBrightness(PWM_FULL);
  } else if (pwmNow == PWM_OFF) {
    setBrightness(PWM_DIM);              // just became dark
  } else if (pwmNow == PWM_FULL && (now - lastMotionMs) >= IDLE_TIMEOUT_MS) {
    setBrightness(PWM_DIM);              // road is clear again
  }
}

void checkFault(unsigned long now) {
  if (!inaOk) return;
  bool lampShouldDraw = (pwmNow > PWM_OFF) && (now - lightOnSinceMs > FAULT_DELAY_MS);
  faultActive = lampShouldDraw && (currentMa < MIN_CURRENT_MA);
}

void uploadData() {
  if (WiFi.status() != WL_CONNECTED) {
    connectWiFi();
    return;
  }
  int brightnessPct = (pwmNow * 100) / PWM_FULL;
  String url = "http://api.thingspeak.com/update?api_key=" + String(TS_API_KEY)
             + "&field1=" + String(ldrValue)
             + "&field2=" + String(brightnessPct)
             + "&field3=" + String(motion ? 1 : 0)
             + "&field4=" + String(currentMa, 1)
             + "&field5=" + String(faultActive ? 1 : 0);
  WiFiClient client;
  HTTPClient http;
  if (http.begin(client, url)) {
    int code = http.GET();
    Serial.print("Upload HTTP code: ");
    Serial.println(code);
    http.end();
  }
}

void printStatus() {
  Serial.print("LDR=");      Serial.print(ldrValue);
  Serial.print(" dark=");    Serial.print(isDark);
  Serial.print(" motion=");  Serial.print(motion);
  Serial.print(" pwm=");     Serial.print(pwmNow);
  Serial.print(" mA=");      Serial.print(currentMa, 1);
  Serial.print(" fault=");   Serial.println(faultActive ? "FAULT" : "ok");
}

void setup() {
  Serial.begin(115200);
  pinMode(PIR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  analogWriteRange(1023);
  setBrightness(PWM_OFF);

  Wire.begin(SDA_PIN, SCL_PIN);
  inaOk = ina219.begin();
  if (!inaOk) Serial.println("INA219 not found - fault check disabled");

  connectWiFi();
  lastMotionMs = millis();
}

void loop() {
  unsigned long now = millis();
```

```
  if (now - lastSampleMs >= SAMPLE_EVERY_MS) {
    lastSampleMs = now;
    readSensors();
    updateLight(now);
    checkFault(now);
  }

  if (now - lastUploadMs >= UPLOAD_EVERY_MS) {
    lastUploadMs = now;
    printStatus();
    uploadData();
  }
}
```

20.1 Code Explanation

The sketch reads the three sensors every 200 ms, applies the darkness hysteresis and the OFF/DIM/FULL decision, checks for a fault from the INA219 current and uploads five values to ThingSpeak every 20 seconds. Lighting never depends on the network.

20.2 Final Submission Checklist

- Student name and roll number on the title page are correct.
- Wi-Fi password and API key are removed from the printed code.
- Photographs and screenshots are inserted in Chapter 19.
- Test tables in Chapters 14 and 15 are filled with real results.
- References follow the college citation style.
- Signatures and college seal are obtained on the certificate page.

21. SYSTEM MODELING USING UML

UML diagrams were used to model the structure and behaviour of the system before implementation. They show the use cases, class structure, working flow, data flow, interactions, activities, data model and deployment.

21.1 Use Case Diagram

The use case diagram shows the actors – Admin/Operator, the Sensor subsystem and Maintenance Staff – and the functions they use, such as detecting light and motion, controlling the street light, monitoring status and handling faults.

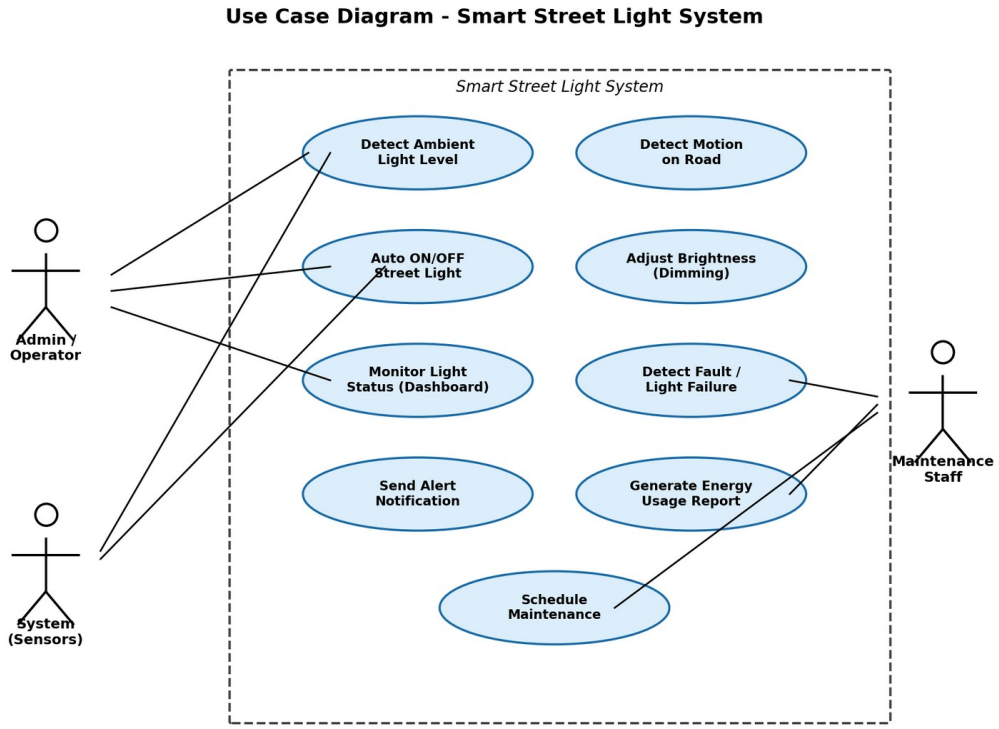


Figure 21.1: Use Case Diagram

21.2 Class Diagram

The class diagram shows the main classes – StreetLight, Sensor, Controller, CommunicationModule, Database/Server and AdminDashboard – with their attributes, methods and relationships.

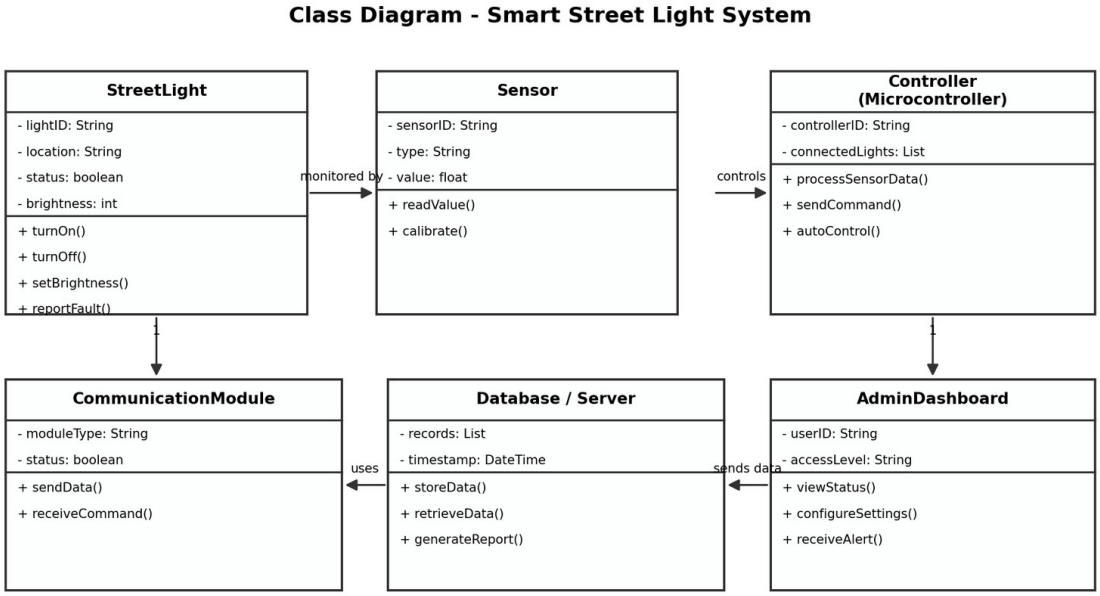


Figure 21.2: Class Diagram

21.3 Flowchart – System Working

The flowchart shows the step-by-step logic: read the LDR, decide whether to turn the light ON, read the motion sensor, adjust brightness and log data. A fault current reading raises an alert at any stage.

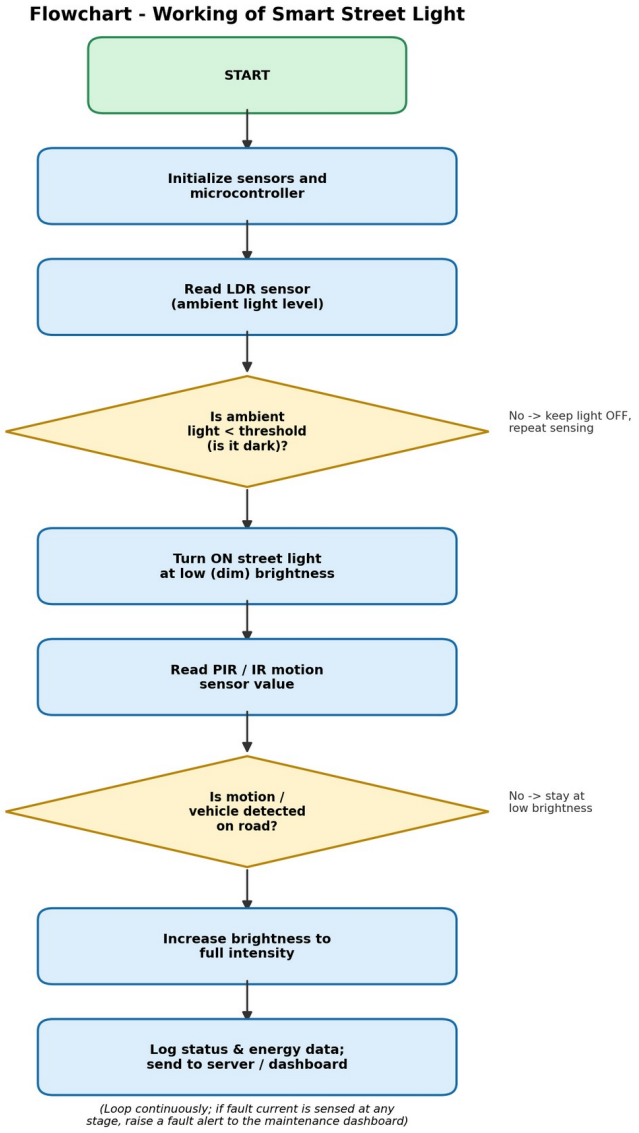


Figure 21.3: Flowchart – Working of the System

21.4 Data Flow Diagram (Level 1)

The Level 1 DFD shows data flowing between the external entities (Sensors, Administrator, Maintenance Staff), the processes (Sense and Acquire Data, Process and Control Light, Monitor and Generate Report, Fault Detection and Alert) and the data stores (Sensor Data Log, Maintenance Records).

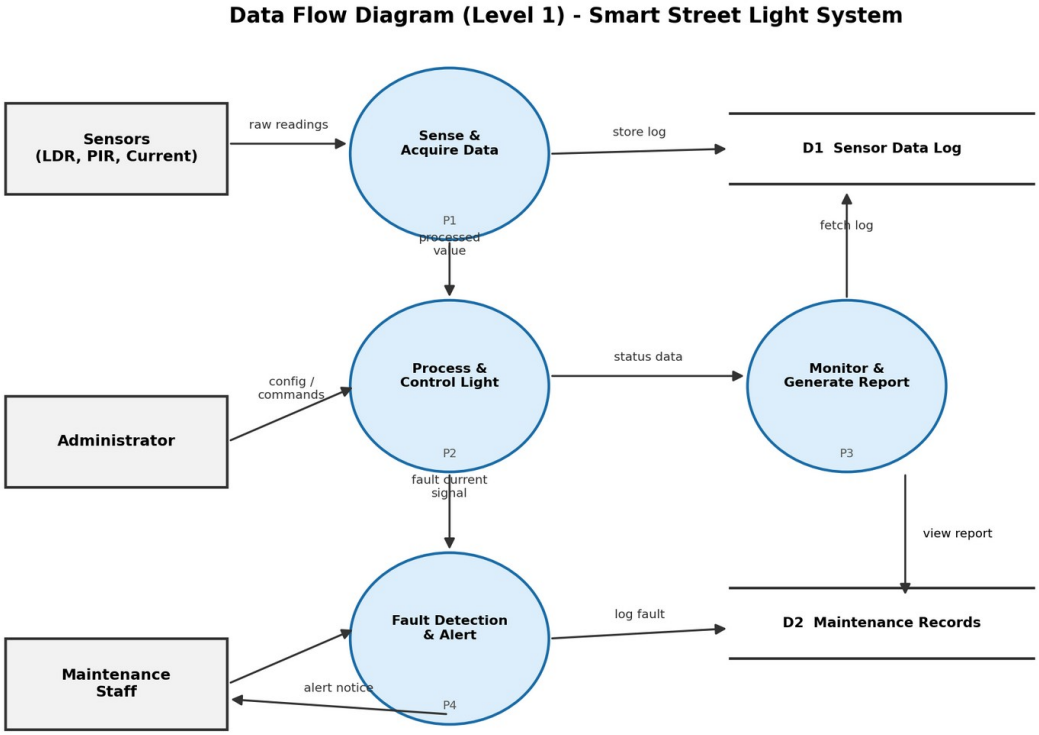


Figure 21.4: Data Flow Diagram (Level 1)

21.5 Sequence Diagram

The sequence diagram shows the order of messages between the sensors, the NodeMCU controller, the LED driver, the cloud and the dashboard during one dark-time cycle with a motion event.

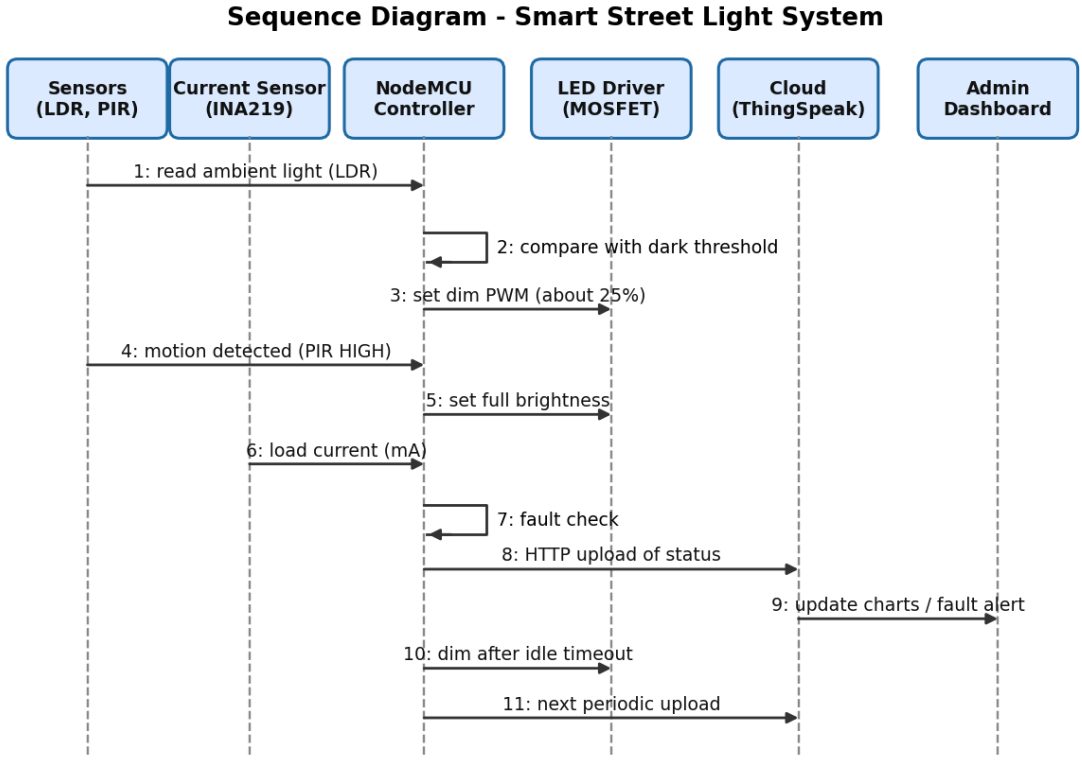


Figure 21.5: Sequence Diagram

21.6 Activity Diagram

The activity diagram shows the controller’s loop with its decision points for darkness, motion and fault.

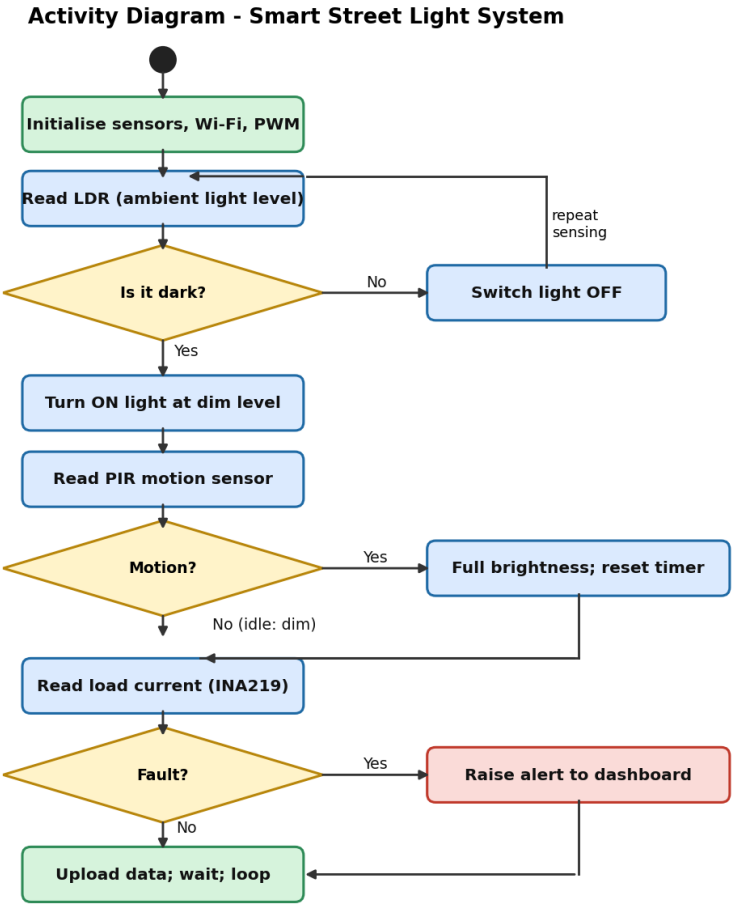
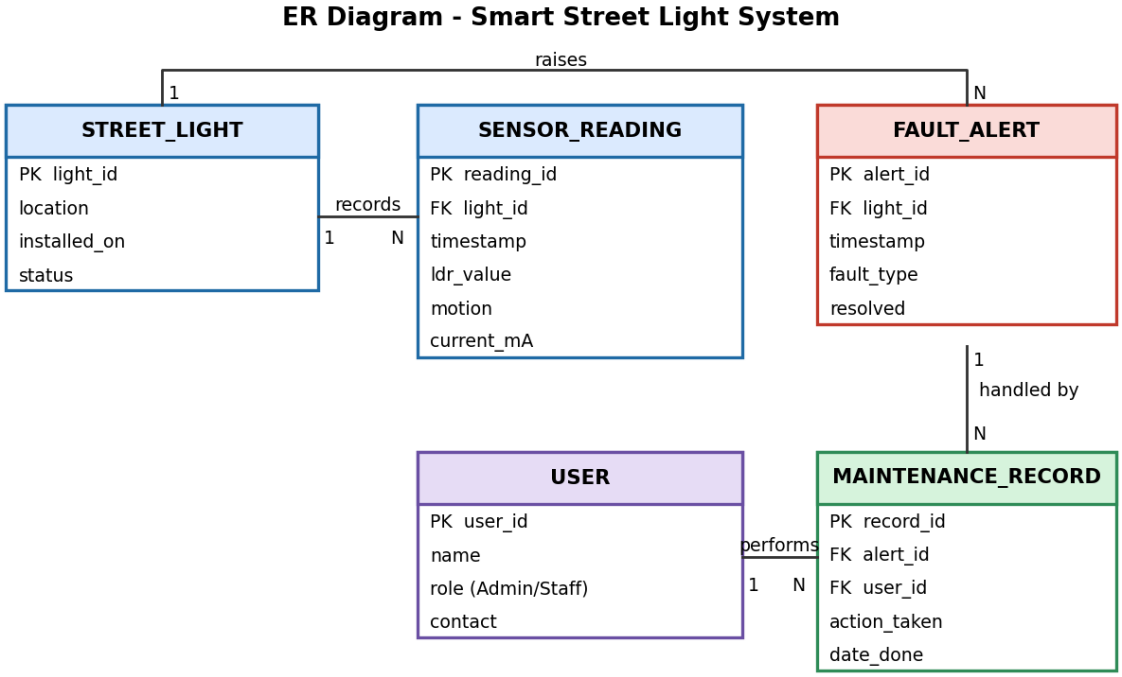


Figure 21.6: Activity Diagram

21.7 ER Diagram

The ER diagram shows the data stored by the cloud or database: street lights, sensor readings, fault alerts, maintenance records and users.



Legend: PK = primary key, FK = foreign key. Cardinality: 1 = one, N = many.

Figure 21.7: ER Diagram

21.8 Deployment Diagram

The deployment diagram shows the pole unit, the Wi-Fi router or GSM gateway, the cloud server and the client devices used by the administrator and maintenance staff.

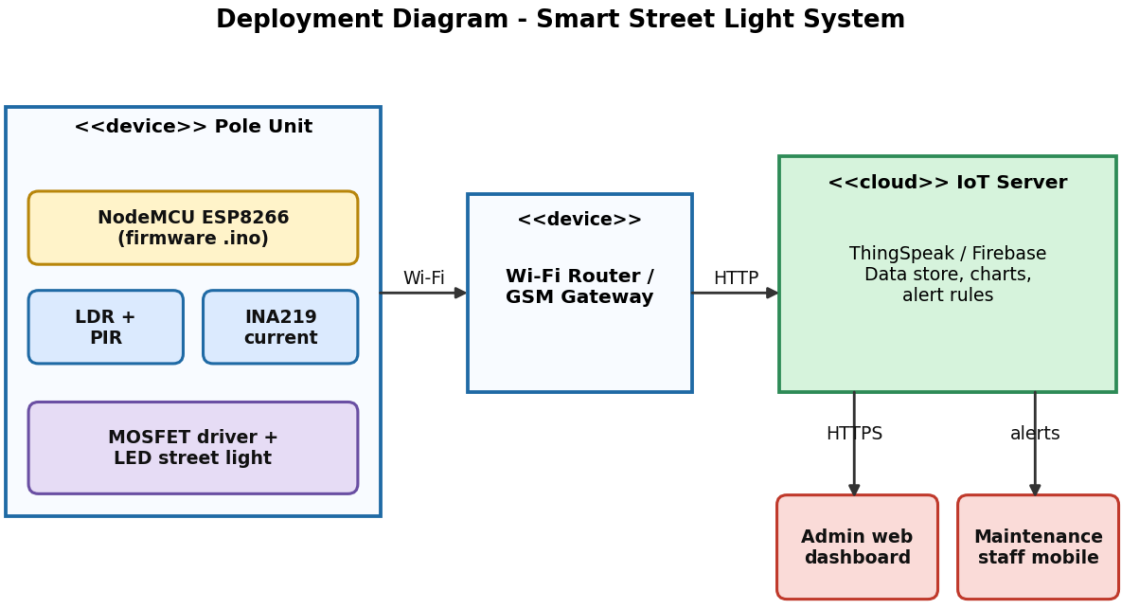


Figure 21.8: Deployment Diagram